```
import numpy as np

marks = np.array([45, 72, 88, 31, 65, 90])
```

```
marks > 80
```

```
array([False, False,  True, False, False,  True])
```

```
marks[marks > 80]
```

```
array([88, 90])
```

```
# np.where(condition, value_if_true, value_if_false)
```

```
marks = np.array([45, 72, 88, 31, 65, 90])

# marks > 70 --> "Good", else "Average"

res = np.where(marks > 75, "Good", "Average")
```

```
res
```

```
array(['Average', 'Average', 'Good', 'Average', 'Average', 'Good'],
      dtype='<U7')
```

```
marks
```

```
array([45, 72, 88, 31, 65, 90])
```

```
# Suppose students scoring below 50 get 5 grace marks.

a = [45, 72, 88, 31, 65, 90]
b = []

for m in a:
  if m< 50:
    b.append(m+5)
  else:
    b.append(m)
```

```
print(b)
```

```
[50, 72, 88, 36, 65, 90]
```

```
marks
```

```
array([45, 72, 88, 31, 65, 90])
```

```
n_m = np.where(marks < 50, marks + 5, marks)
```

```
n_m
```

```
array([50, 72, 88, 36, 65, 90])
```

```
# Suppose products costing more than ₹1,000 receive a 10% discount.

prices = np.array([500, 1200, 800, 2500, 1500])
n_p = np.where(prices > 1000, prices*0.9, prices)
n_p
```

```
array([ 500., 1080.,  800., 2250., 1350.])
```

```
salary = np.array([25000, 45000, 75000, 32000, 90000])

# Salary >= 50000 → High
# Salary < 50000 → Standard
```

```
s_c = np.where(salary >= 50000, "High", "Standard")
s_c
```

```
array(['Standard', 'Standard', 'High', 'Standard', 'High'], dtype='<U8')
```

```
marks
```

```
array([45, 72, 88, 31, 65, 90])
```

```
# 90 or above → Excellent
# 50–89 → Pass
# Below 50 → Fail
```

```
n_m = np.where(marks >= 90,"Excellent", np.where(marks <50, "Fail", "Pass") )
```

```
n_m
```

```
array(['Fail', 'Pass', 'Pass', 'Fail', 'Pass', 'Excellent'], dtype='<U9')
```

```
sales = np.array([950000, 650000, 300000, 800000, 450000])


Categories:

>= 800000 → "Top Performer"

else if >= 500000 → "Good Performer"

otherwise → "Needs Improvement"
```

```
sales = np.array([950000, 650000, 300000, 800000, 450000])
```

```
n_s = np.where(sales >= 800000, "Top Performer", np.where(sales >=500000, "Good Performer", "Needs Improvement") )
n_s
```

```
array(['Top Performer', 'Good Performer', 'Needs Improvement',
       'Top Performer', 'Needs Improvement'], dtype='<U17')
```

```
Suppose a loan applicant must satisfy both:

salary >= ₹50,000
credit score >= 700

then approve, else reject

salary = np.array([60000, 40000, 75000, 55000, 30000])
credit_score = np.array([720, 750, 680, 710, 800])
```

```
salary = np.array([60000, 40000, 75000, 55000, 30000])
credit_score = np.array([720, 750, 680, 710, 800])
```

```
approve = (salary >= 50000) & (credit_score >= 700)
approve
```

```
array([ True, False, False,  True, False])
```

```
ans = np.where(approve, "Approve", "Reject")
ans
```

```
array(['Approve', 'Reject', 'Reject', 'Approve', 'Reject'], dtype='<U7')
```

```
Suppose a customer receives free shipping if:

order value >= ₹1,000
OR
customer is a premium customer.


order_value = np.array([500, 1200, 700, 800, 1500])
premium = np.array([False, False, True, False, True])
```

```
order_value = np.array([500, 1200, 700, 800, 1500])
premium = np.array([False, False, True, False, True])
```

```
free = (order_value > 1000) | (premium == True)
free
```

```
array([False,  True,  True, False,  True])
```

```
is_free = np.where(free, "Free", "Paid")
is_free
```

```
array(['Paid', 'Free', 'Free', 'Paid', 'Free'], dtype='<U4')
```

```
students = np.array([
    [101, 78, 85, 92],
    [102, 45, 55, 60],
    [103, 91, 88, 95],
    [104, 62, 70, 68],
    [105, 35, 42, 50]
])

Student_ID | Python | SQL | ML

# calculate avg marks of each student
```

```
students = np.array([
    [101, 78, 85, 92],
    [102, 45, 55, 60],
    [103, 91, 88, 95],
    [104, 62, 70, 68],
    [105, 35, 42, 50]
])
```

```
Start coding or generate with AI.
```

```
avg_m = np.mean(students[:, 1:], axis = 1)
```

```
# Let's classify students as:

# Average >= 85 → Excellent
# Average >= 60 → Good
# Average >= 50 → Pass
# Otherwise → Fail
```

```
avg_m
```

```
array([85.        , 53.33333333, 91.33333333, 66.66666667, 42.33333333])
```

```
ans = np.where(avg_m >= 85, "Excellent", np.where(avg_m >= 60, "Good", np.where(avg_m >= 50, "Pass", "Fail")))
ans
```

```
array(['Excellent', 'Pass', 'Excellent', 'Good', 'Fail'], dtype='<U9')
```

```
A business records revenue for 12 months:

revenue = np.array([
    120000, 135000, 128000, 145000,
    150000, 138000, 160000, 172000,
    155000, 148000, 180000, 195000
])

Find:

Average monthly revenue
Highest revenue
Lowest revenue
Month index with highest revenue
Month index with lowest revenue
```

```
Total annual revenue
Difference between highest and lowest revenue
```

```
revenue = np.array([
    120000, 135000, 128000, 145000,
    150000, 138000, 160000, 172000,
    155000, 148000, 180000, 195000
])
```

```
revenue
```

```
array([120000, 135000, 128000, 145000, 150000, 138000, 160000, 172000,
       155000, 148000, 180000, 195000])
```

```
avg_r = np.round(np.mean(revenue),2)
avg_r
```

```
np.float64(152166.67)
```

```
max_r = np.max(revenue)
max_r
```

```
np.int64(195000)
```

```
# Lowest revenue
min_r = np.min(revenue)
print(min_r)
```

```
120000
```

```
# Month index with highest revenue
np.argmax(revenue)
```

```
np.int64(11)
```

```
np.argmin(revenue)
```

```
np.int64(0)
```

```
# Total annual revenue
np.sum(revenue)
```

```
np.int64(1826000)
```

```
# show the quaterly total revenue.

qtr = []
qtr.append(revenue[:3])
qtr.append(revenue[3:6])
qtr.append(revenue[6:9])
qtr.append(revenue[9:12])
```

```
qtr
```

```
[array([120000, 135000, 128000]),
 array([145000, 150000, 138000]),
 array([160000, 172000, 155000]),
 array([148000, 180000, 195000])]
```

```
q_n = np.array(qtr)
```

```
np.sum(q_n, axis = 1)
```

```
array([383000, 433000, 487000, 523000])
```

```
# Difference between highest and lowest revenue

np.max(revenue) - np.min(revenue)
```

```
np.int64(75000)
```

```
The following array contains sales made by 4 salespeople across 3 months.

sales = np.array([
    [120, 150, 180],
    [200, 170, 160],
    [100, 140, 130],
    [180, 190, 210]
])

Rows represent salespeople and columns represent months.

Find:

Total sales by each salesperson
Average sales by each salesperson
Total sales for each month
Average sales for each month
Highest sales achieved by each salesperson
```

```
sales = np.array([
    [120, 150, 180],
    [200, 170, 160],
    [100, 140, 130],
    [180, 190, 210]
])
```

```
# Total sales by each salesperson

np.sum(sales, axis = 1)
```

```
array([450, 530, 370, 580])
```

```
# Average sales by each salesperson
np.round(np.mean(sales, axis = 1),2)
```

```
array([150.  , 176.67, 123.33, 193.33])
```

```
# Total sales for each month
np.sum(sales, axis = 0)
```

```
array([600, 650, 680])
```

```
# Average sales for each month
np.mean(sales, axis = 0)
```

```
array([150. , 162.5, 170. ])
```

```
# Highest sales achieved by each salesperson

np.max(sales, axis = 1)
```

```
array([180, 200, 140, 210])
```

HW

```
Consider:

employees = np.array([
    [101, 35000, 2],
    [102, 48000, 4],
    [103, 62000, 6],
    [104, 40000, 3],
    [105, 75000, 8],
    [106, 55000, 5]
])

Columns represent:

Employee ID | Salary | Experience

Find:
```

```
Employees earning more than ₹50,000.
Employees with more than 4 years of experience.
Employees earning more than ₹50,000 AND having more than 4 years of experience.
Average salary of employees satisfying condition 3.
Employee with the highest salary.
Employees earning below the median salary.
```

```
employees = np.array([
    [101, 35000, 2],
    [102, 48000, 4],
    [103, 62000, 6],
    [104, 40000, 3],
    [105, 75000, 8],
    [106, 55000, 5]
])
```

```
employees[employees[:,1] > 50000]
```

```
array([[  103, 62000,     6],
       [  105, 75000,     8],
       [  106, 55000,     5]])
```

```
employees[:,1] > 50000
```

```
array([False, False,  True, False,  True,  True])
```

```
Start coding or generate with AI.
```